

Chuyên đề: Kỹ Thuật Xử Lý Ngoại Tuyến & Quét Sự Kiện (Offline Processing & Sweep-Line Techniques)

(Offline Query Processing, Right-Endpoint Sweep, Mo's Algorithm, Time Divide & Conquer, Parallel Binary Search)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Tư Duy Cốt Lõi: Bản Chất của Xử Lý Ngoại Tuyến (Offline vs Online)

1.1. Phân Biệt Bài Toán Online và Offline

Trong lập trình thi đấu, các bài toán có truy vấn (Queries) được chia làm hai loại cơ chế:

1. **Trực tuyến (Online Processing):**
2. Đề bài bắt buộc phải trả lời truy vấn thứ i mới được phép đọc truy vấn thứ $i + 1$.
3. Thông thường, giám khảo mã hóa dữ liệu nhập: $L' = (L \oplus last_ans) \bmod N$, ngăn chặn việc đọc trước toàn bộ đề bài.
4. Bắt buộc phải sử dụng các cấu trúc dữ liệu phức tạp: Persistent Segment Tree (Cây phân đoạn bền bỉ), Link-Cut Tree, Treap.
5. **Ngoại tuyến (Offline Processing):**
6. Đề bài cung cấp trước toàn bộ Q truy vấn. Kết quả của truy vấn sau **hoàn toàn không phụ thuộc** vào câu trả lời của truy vấn trước.
7. Ta có toàn quyền: **Lưu trữ toàn bộ Q truy vấn, sắp xếp lại theo một trật tự có lợi nhất**, xử lý tập trung rồi xuất kết quả theo thứ tự xuất hiện ban đầu.

 **Quan trọng:**

****Quy tắc vàng:****

"Nếu đề bài không bắt buộc Online, hãy luôn ưu tiên tư duy Offline trước tiên!"

Xử lý Offline biến một bài toán cập nhật động cực khó thành một chuỗi các thao tác tĩnh, giảm độ phức tạp

tập từ $O(Q \log^2 N)$ với bộ nhớ khổng lồ xuống còn $O((N + Q) \log N)$ với mã nguồn ngắn gọn và hằng số thực thi siêu tốc.

2. Kỹ Thuật 1: Quét Cận Phải (Right-Endpoint Sweep)

Đây là kỹ thuật Offline phổ biến nhất trong các kỳ thi ICPC / OLP.

2.1. Bản Chất Kỹ Thuật

Xét bài toán: Cho mảng A gồm N số, trả lời Q truy vấn trên đoạn $[L, R]$.

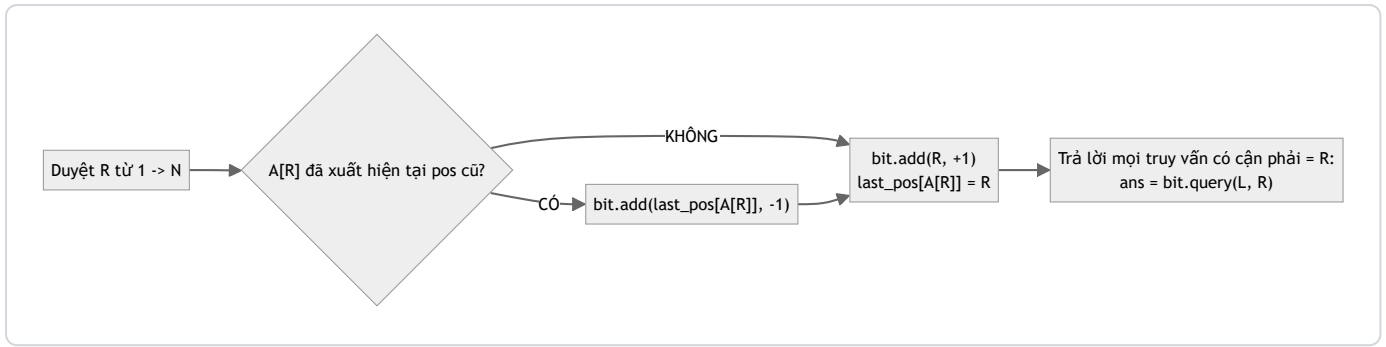
Thay vì xử lý từng đoạn độc lập:

1. Lưu lại toàn bộ các truy vấn vào một danh sách kèm chỉ số gốc của nó.
2. Gom các truy vấn theo cận phải R : `queries_at[R].push_back({L, query_id})`.
3. Cho một biến R chạy từ 1 đến N (quét từ trái sang phải):
 - Khi phần tử $A[R]$ được nạp vào, ta cập nhật ảnh hưởng của nó lên cấu trúc dữ liệu (Fenwick Tree hoặc Segment Tree).
 - Ngay tại bước R , toàn bộ các truy vấn kết thúc tại R đều đã có **đầy đủ thông tin của tiền tố** $[1, R]$.
 - Ta chỉ cần truy vấn đoạn $[L, R]$ trên Fenwick Tree và ghi nhận kết quả vào `ans[query_id]`.

2.2. Case Study Điển Hình: Đếm Số Phần Tử Phân Biệt (CSES 1734 — Distinct Values Queries)

- **Yêu cầu:** Đếm số lượng giá trị phân biệt trong đoạn $A[L \dots R]$.
- **Nhận xét quan trọng:**
Nếu một giá trị v xuất hiện nhiều lần trong tiền tố $[1, R]$, chỉ có **lần xuất hiện cuối cùng (gần R nhất)** mới có khả năng đóng góp vào kết quả của một đoạn $[L, R]$. Các lần xuất hiện trước đó đã bị che khuất hoàn toàn!
- **Cơ chế cập nhật:**
 - Duy trì mảng `last_pos[v]` lưu vị trí gần nhất của giá trị v .
 - Khi duyệt tới R :
 - Nếu giá trị $A[R]$ đã từng xuất hiện tại $p = \text{last_pos}[A[R]]$: Xóa đóng góp của vị trí cũ bằng cách gán `bit.add(p, -1)`.
 - Đóng góp vị trí mới: `bit.add(R, +1)`.
 - Cập nhật `last_pos[A[R]] = R`.
- Với mọi truy vấn $[L, R]$, số phần tử phân biệt chính là tổng trên Fenwick Tree từ L đến R :

$$\text{ans} = \text{bit.query}(L, R) = \text{bit.query}(R) - \text{bit.query}(L - 1)$$



2.3. Bảng Trace Step-by-Step cho CSES 1734

Xét mảng $A = [3, 2, 3, 1, 2]$ ($N = 5$) với 3 truy vấn:

- $Q_1 : [1, 3]$ (Đoạn $[3, 2, 3]$)
- $Q_2 : [2, 4]$ (Đoạn $[2, 3, 1]$)
- $Q_3 : [1, 5]$ (Đoạn $[3, 2, 3, 1, 2]$)

Bước R	Phần tử $A[R]$	Vị trí cũ p	Thao tác trên Fenwick Tree	Trạng thái mảng BIT sau bước R	Truy vấn được trả lời tại R	Kết quả tính toán
1	3	Chưa có	add(1, +1)	[1, 0, 0, 0, 0]	—	—
2	2	Chưa có	add(2, +1)	[1, 1, 0, 0, 0]	—	—
3	3	$p = 1$	add(1, -1) , add(3, +1)	[0, 1, 1, 0, 0]	$Q_1 : [1, 3]$	$\text{sum}(1 \dots 3) = 0 + 1 + 1 = \mathbf{2}$
4	1	Chưa có	add(4, +1)	[0, 1, 1, 1, 0]	$Q_2 : [2, 4]$	$\text{sum}(2 \dots 4) = 1 + 1 + 1 = \mathbf{3}$
5	2	$p = 2$	add(2, -1) , add(5, +1)	[0, 0, 1, 1, 1]	$Q_3 : [1, 5]$	$\text{sum}(1 \dots 5) = 0 + 0 + 1 + 1 + 1 = \mathbf{3}$

Nhận xét: Toàn bộ thuật toán chạy trong đúng $O(N \log N + Q \log N)$, mã nguồn chưa đầy 40 dòng!

3. Kỹ Thuật 2: Thuật Toán Mo (Mo's Algorithm)

Khi bài toán trên đoạn $[L, R]$ **không có tính chất trừ dồn** (không dùng được Fenwick/Segment Tree) nhưng lại **rất dễ mở rộng hoặc thu hẹp 1 phần tử** trong $O(1)$:

- Thêm phần tử vào đoạn: $O(1)$.
- Xóa phần tử khỏi đoạn: $O(1)$.

Thuật toán Mo cho phép trả lời Q truy vấn tính trong tổng thời gian $O((N + Q)\sqrt{N})$ bằng kỹ thuật phân rã căn bậc hai (Square Root Decomposition) trên các truy vấn.

3.1. Phân Khối & Trật Tự Sắp Xếp Truy Vấn

1. Chia mảng A thành các khối kích thước B : Khối của chỉ số L là $\text{block}(L) = \lfloor \frac{L}{B} \rfloor$.
2. Sắp xếp Q truy vấn theo tiêu chí:
3. Ưu tiên 1: Khối của L tăng dần ($\text{block}(L_1) < \text{block}(L_2)$).
4. Ưu tiên 2: Cận phải R tăng dần.

```
int B = max(1, (int)(n / sqrt(q)));
struct Query {
    int l, r, id;
    bool operator<(const Query& other) const {
        int b1 = l / B, b2 = other.l / B;
        if (b1 != b2) return b1 < b2;
        return (b1 & 1) ? r < other.r : r > other.r; // Tối ưu Odd-Even Sort!
    }
};
```

3.2. Chứng Minh Độ Phức Tạp Chặt Chẽ của Mo

Giả sử ta duy trì hai con trỏ `curL` và `curR` đại diện cho đoạn hiện tại đang được tính toán.

1. **Sự di chuyển của con trỏ R :**
2. Trong cùng một khối L : Các giá trị R được sắp xếp tăng dần từ $1 \dots N$. Do đó, trong suốt một khối, R chỉ tăng đơn điệu từ đầu đến cuối mảng $\implies$ di chuyển tối đa N bước.
3. Có tổng cộng $\frac{N}{B}$ khối $\implies$ Tổng số bước di chuyển của R qua tất cả các khối là:

$$\text{Total}_R = \frac{N}{B} \cdot N = \frac{N^2}{B}$$

4. **Sự di chuyển của con trỏ L :**
5. Khi chuyển giữa hai truy vấn liên tiếp: Cả hai đầu L đều nằm trong cùng một khối độ dài $B \implies$ Mỗi truy vấn, con trỏ L chỉ di chuyển tối đa B bước.
6. Có Q truy vấn $\implies$ Tổng số bước di chuyển của L là:

$$\text{Total}_L = Q \cdot B$$

7. **Cân bằng tham số B :**

$$\text{Total} = \frac{N^2}{B} + Q \cdot B$$

Theo bất đẳng thức Cauchy (AM-GM), tổng đạt giá trị nhỏ nhất khi $\frac{N^2}{B} = Q \cdot B \implies B^2 = \frac{N^2}{Q} \implies B = \frac{N}{\sqrt{Q}}$.

Khi đó, tổng số bước di chuyển là:

$$\text{Complexity} = O\left(N\sqrt{Q} + Q \cdot \frac{N}{\sqrt{Q}}\right) = O(N\sqrt{Q})$$

(Nếu $N \approx Q$, độ phức tạp đạt $O(N\sqrt{N})$).

💡 Mẹo / Tối ưu:

****Tối ưu hóa Odd-Even Sort (Sắp xếp Lượn Sóng):****

Khi đổi sang khối tiếp theo, thay vì để con trỏ R phải chạy ngược từ N về 0, ta sắp xếp R tăng dần ở khối chẵn và giảm dần ở khối lẻ ($(b1 \& 1) ? r < other.r : r > other.r$).

Kỹ thuật này giúp con trỏ R di chuyển theo đường ziczac liên tục, ****giảm đúng 50% số bước di chuyển của R , tăng tốc chương trình lên gấp đôi!**

4. Kỹ Thuật 3: Chia Để Trị Theo Thời Gian & Rollback DSU (Dynamic Connectivity)

Bài toán Đồ thị động Offline (Offline Dynamic Connectivity):

Cho đồ thị rỗng gồm N đỉnh. Có Q sự kiện xảy ra theo dòng thời gian từ $1 \dots Q$:

1. Thêm cạnh (u, v) vào đồ thị.
2. Xóa cạnh (u, v) khỏi đồ thị.
3. Hỏi hai đỉnh u và v có liên thông hay không?

4.1. Khái Niệm Khoảng Tồn Tại Của Cạnh

Một cạnh (u, v) được thêm vào tại thời điểm t_{in} và bị xóa tại thời điểm t_{out} .

Nghĩa là cạnh (u, v) **chỉ tồn tại và có hiệu lực trong khoảng thời gian** $[t_{in}, t_{out} - 1]$.

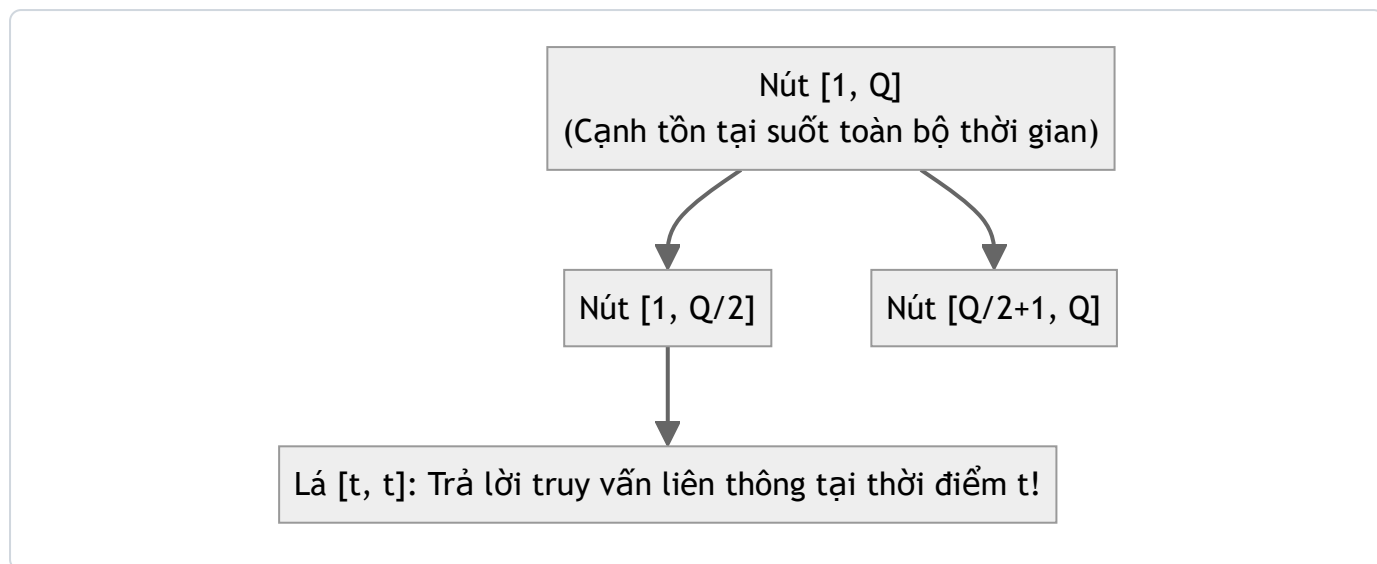
(Nếu cạnh không bị xóa cho tới hết contest, $t_{out} = Q + 1$).

Thời gian:	1	2	3	4	5	6	7	8	
Cạnh (1, 2):	[-----]								($t_{in} = 1, t_{out} = 5$)
Cạnh (2, 3):			[-----]						($t_{in} = 3, t_{out} = 7$)
Truy vấn:	?		?		?		?		

4.2. Dựng Segment Tree Trên Trục Thời Gian $1 \dots Q$

Ta dựng một Cây Phân Đoạn quản lý đoạn thời gian $[1, Q]$:

- Mỗi khoảng tồn tại $[t_{in}, t_{out} - 1]$ của một cạnh được phân rã thành tối đa $2 \log_2 Q$ nút trên Segment Tree (tương tự như Range Update thông thường).
- Tại mỗi nút của Segment Tree, ta lưu một danh sách các cạnh phủ lên đoạn đó: `vector<Edge> tree[id]`.



4.3. Duyệt DFS Kết Hợp DSU Hoàn Tác (Rollback DSU)

1. Bắt đầu DFS từ gốc của Segment Tree (thời gian $[1, Q]$).
2. Khi bước vào nút `id`: Duyệt qua toàn bộ danh sách cạnh của nút đó và thực hiện `dsu.unite(u, v)`. Ghi lại số lượng phép gộp đỉnh đã thực hiện.
3. Nếu chạm tới nút lá ($l = r$): Nếu tại thời điểm l có truy vấn hỏi liên thông, ta gọi `dsu.same(u, v)` và lưu kết quả.
4. Ngược lại ($l < r$): Đệ quy thăm con trái, sau đó thăm con phải.
5. **Backtracking (Quay lui)**: Trước khi rời khỏi nút `id` để trở về cha, ta gọi `dsu.rollback()` đúng bằng số phép gộp đã làm ở bước 2 để phục hồi nguyên trạng cây DSU!

📌 Quan trọng:

****Rollback DSU vs Path Compression:****

Trong Rollback DSU, ta ****KHÔNG ĐƯỢC dùng Nén đường đi (Path Compression)**** vì nén đường đi phá vỡ cấu trúc cây, làm mất khả năng hoàn tác.

Ta chỉ sử dụng ****Gộp theo kích thước (Union by Size)****. Độ sâu cây luôn $\leq \log_2 N$.

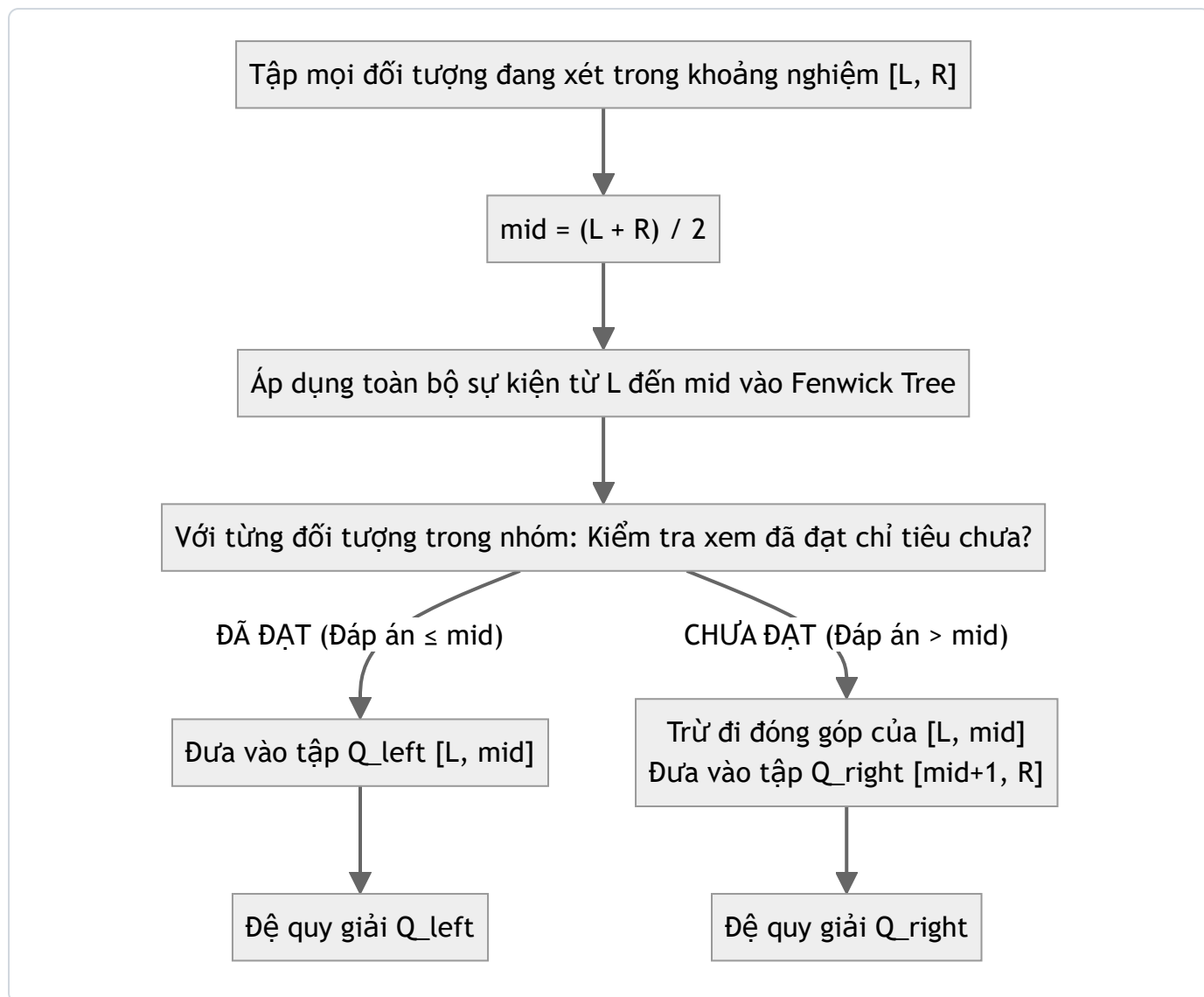
Mỗi thao tác gộp và hoàn tác mất $O(\log N)$.

****Tổng độ phức tạp toàn bài:**** $O(Q \log Q \cdot \log N)$ — Cực kỳ mạnh mẽ và thanh lịch!

5. Kỹ Thuật 4: Chặt Nhị Phân Song Song (Parallel Binary Search)

Bài toán: Cho N đối tượng và Q sự kiện xảy ra tuần tự. Mỗi đối tượng i cần đạt được một chỉ tiêu K_i . Ta cần tìm **thời điểm sớm nhất** mà đối tượng i đạt chỉ tiêu.

- **Cách tiếp cận ngây thơ**: Chặt nhị phân thời điểm cho từng đối tượng $i \in [1, N]$. Mỗi bước kiểm tra thời điểm mid cần mô phỏng lại các sự kiện từ $1 \dots mid$ bằng Fenwick Tree ($O(Q \log M)$).
Tổng độ phức tạp: $N \times O(Q \log M \log Q) = O(NQ \log^2) \implies$ **TLE chắc chắn!**
- **Chặt nhị phân song song (PBS)**: Chặt nhị phân đồng thời cho **tất cả N đối tượng cùng một lúc!**



Phân Tích Độ Phức Tạp của PBS

- Tại mỗi tầng đệ quy, mỗi sự kiện chỉ được thêm vào Fenwick Tree đúng 1 lần.
- Mỗi đối tượng chỉ được kiểm tra đúng 1 lần.
- Chi phí mỗi tầng: $O((M + N) \log M)$.
- Cây nhị phân phân đôi không gian tìm kiếm có độ sâu đúng $\log_2 Q$.
- **Tổng thời gian:** $O((M + N) \log M \log Q)$, chạy mượt mà trong 0.3 giây với $N, M, Q = 3 \times 10^5$!

6. Bảng So Sánh Các Kỹ Thuật Xử Lý Ngoại Tuyến

Kỹ thuật	Điều kiện Áp dụng	Cấu trúc Dữ liệu Kết hợp	Độ Phức Tạp Thời Gian	Độ Phức Tạp Bộ Nhớ
Quét Cận Phải (Sweep-line)	Truy vấn đoạn tĩnh, có tính chất "lần xuất hiện gần nhất"	Fenwick Tree / Segment Tree	$O((N + Q) \log N)$	$O(N + Q)$

Kỹ thuật	Điều kiện Áp dụng	Cấu trúc Dữ liệu Kết hợp	Độ Phức Tạp Thời Gian	Độ Phức Tạp Bộ Nhớ
Quét Ngưỡng (Weight Sweep)	Đồ thị tĩnh, truy vấn có điều kiện lọc trọng số $W \leq K$	Disjoint Set Union (DSU)	$O((E + Q) \log(E + Q))$	$O(E + Q)$
Thuật toán Mo cơ bản	Truy vấn đoạn tĩnh, thêm/xóa 1 phần tử mất $O(1)$	Mảng tần số / Hash map	$O((N + Q)\sqrt{N})$	$O(N + Q)$
Thuật toán Mo có cập nhật	Truy vấn đoạn có cập nhật điểm đơn lẻ ($A[p] = v$)	Mảng tần số + Dòng thời gian	$O(N^{5/3})$	$O(N + Q)$
Chia để trị thời gian	Đồ thị động Offline: Thêm cạnh, xóa cạnh, hỏi liên thông	Segment Tree + Rollback DSU	$O(Q \log Q \log N)$	$O(Q \log Q)$
Chặt nhị phân song song	N bài toán tìm kiếm nhị phân độc lập trên cùng dòng sự kiện	Fenwick Tree / Segment Tree	$O((N + Q) \log N \log(\text{Time}))$	$O(N + Q)$

7. Cạm Bẫy Phòng Thi & Kỹ Thuật Tối Ưu

1. Khôi phục thứ tự xuất ban đầu:

Khi xử lý Offline, các truy vấn bị đảo lộn trật tự. Luôn lưu trường `id` trong cấu trúc truy vấn và ghi kết quả vào mảng `ans[q.id]`, sau đó in lần lượt từ $1 \dots Q$.

2. Kích thước khối B trong Mo's Algorithm:

Không mặc định gán $B = \sqrt{N}$. Công thức tối ưu nhất là $B = \max(1, \lfloor \frac{N}{\sqrt{Q}} \rfloor)$. Nếu $Q \ll N$ hoặc $Q \gg N$, đặt sai B có thể làm chương trình chạy chậm gấp 5-10 lần và bị TLE.

3. Cạm bẫy Rollback DSU:

Tuyệt đối không áp dụng Nén đường đi (`parent[u] = find(parent[u])`) trong DSU có tính năng Rollback, vì nó làm thay đổi nhiều con trỏ cùng lúc mà ngăn xếp không thể theo dõi hết.

4. Giới hạn bộ nhớ trong Segment Tree thời gian:

Mỗi cạnh được đẩy vào tối đa $2 \log Q$ nút của Segment Tree. Nếu có $Q = 2 \times 10^5$ cạnh, tổng số phần tử trong các `vector` cạnh là $2 \times 10^5 \times 36 \approx 7 \times 10^6$ phần tử ≈ 50 MB, hoàn toàn an toàn trong giới hạn 256MB.

8. Tài liệu Tham khảo & Đọc thêm

1. **USACO Guide (Platinum Section):**
2. [Sweep-Line & Offline Processing](#) — Xử lý truy vấn ngoại tuyến kết hợp cây chỉ số.
3. **VNOI Wiki:**
4. [Thuật toán Mo](#) — Xử lý ngoại tuyến với kỹ thuật chia khối căn bậc hai.

5. **Codeforces Community:**

6. [Divide and Conquer on Offline Dynamic Connectivity](#) — Chia để trị thời gian kết hợp Rollback DSU.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.